

Product Requirements Document (PRD)

Project: IntelliRank — AI-Powered Candidate Discovery & Ranking Engine

Hackathon: RedRob — Data & AI Challenge: Intelligent Candidate Discovery **Deadline:** July 2, 2026, 11:59 PM IST

1. Problem Statement

Recruiters screen hundreds of candidate profiles per role. Traditional ATS filtering is keyword-based — it matches literal terms in a resume/profile against a JD and misses candidates whose *actual fit* doesn't surface as exact text overlap. This causes two failure modes:

- **False negatives:** strong candidates get filtered out because their resume uses different terminology than the JD ("ML Engineer" vs "Applied Scientist", "Node.js" vs "JavaScript backend").
- **False positives:** keyword-stuffed profiles rank highly despite poor real-world fit, because keyword match has no concept of context, depth, or behavioral signal.

The opportunity: build a system that reasons about a JD the way a senior recruiter would — understanding role *intent*, then triangulating fit across three data layers: who the candidate **is** (profile/skills), what their **career trajectory** shows (metadata), and how they **behave** in the wild (activity/engagement signals) — and produces a ranked, explainable shortlist.

2. Goals & Non-Goals

Goals

Goal	Why it matters for judging
Parse unstructured JD text into structured role intent	Directly addresses "Deep Job Understanding"
Rank candidates using semantic similarity, not keyword match	Directly addresses "Contextual Relevance"
Fuse profile + career metadata + behavioral signals into one score	Directly addresses "Signal Integration"

Goal	Why it matters for judging
Sub-few-second ranking latency per JD for a batch of candidates	Directly addresses "lightning-fast"
Per-candidate human-readable justification for rank	Builds recruiter trust; demonstrates "expertly ranked" isn't a black box
Clean, working, demoable repo + correctly formatted submission artifacts	Submission compliance is binary — must not be missed

Non-Goals (explicitly out of scope for this PoC)

- Production-grade auth, multi-tenant SaaS, ATS integration
- Actual candidate outreach / messaging
- Full bias/fairness audit pipeline (we will note fairness considerations in the deck, not build a full audit suite)
- Real-time scraping of LinkedIn/GitHub (we work off the provided dataset only)

3. Target User

Primary persona: A recruiter/hiring manager who pastes or selects a JD and needs a trustworthy, ranked shortlist of top-N candidates from a pool, with enough explanation to defend the shortlist to a hiring manager.

Secondary "user": The hackathon judge, who will read the PDF deck, run the GitHub repo, and inspect the ranked XLSX output. Every deliverable should be judge-legible in under 5 minutes.

4. User Stories

1. *As a recruiter*, I paste a raw job description and get back a ranked list of candidates within seconds, without manually defining filters.
2. *As a recruiter*, I can see **why** candidate #1 outranks candidate #5 — not just a score, but a 1-2 line rationale.
3. *As a recruiter*, I can see a breakdown of the score (semantic fit / career fit / behavioral signal / skill match) so I know which dimension drove the rank.
4. *As a recruiter*, I can adjust how much weight behavioral signals vs. career history get, depending on role seniority (e.g., for a senior role, career trajectory might matter more

than GitHub activity).

5. *As a judge*, I can clone the repo, run one command, and get a ranked XLSX out the other end using the provided dataset.

5. Functional Requirements

ID	Requirement
FR1	System accepts a raw-text job description as input
FR2	System extracts structured role requirements from the JD: must-have skills, nice-to-have skills, min/max experience, seniority level, domain, education requirements (via LLM-assisted parsing)
FR3	System loads the candidate dataset and normalizes/parses each profile into a structured representation
FR4	System generates semantic embeddings for both JD and candidate profiles and computes a semantic similarity score
FR5	System computes a career-metadata fit score (experience match, role progression, tenure stability, domain/industry alignment)
FR6	System computes a behavioral/activity signal score from whatever engagement/activity fields exist in the dataset (e.g. platform activity, endorsements, recency of activity, profile completeness)
FR7	System combines FR4–FR6 into a single weighted final score (0–100) and produces a rank order
FR8	System generates a short natural-language explanation per ranked candidate
FR9	System exports the ranked shortlist to XLSX in the submission-required format
FR10	System exposes a UI to paste a JD, trigger ranking, and view results with score breakdown and explanation
FR11	Scoring weights are configurable (not hardcoded), so the same engine can be re-tuned per role type

6. Non-Functional Requirements

- **Performance:** Ranking a candidate pool of ~500–1000 profiles against one JD should complete in well under 10 seconds end-to-end (excluding one-time embedding pre-computation, which can be cached/batched ahead of time).
 - **Explainability:** Every score must be decomposable — no opaque single number without a breakdown.
 - **Determinism where possible:** Given the same JD + dataset + weights, results should be reproducible (cache embeddings, pin model versions, avoid relying on non-deterministic LLM sampling for the score itself — only use LLM generation for the human-readable explanation text).
 - **Robustness:** Must handle missing fields gracefully (not every candidate will have every signal populated) — missing signals should degrade gracefully, not crash or zero out unfairly.
 - **Portability:** Should run from a clean clone with a documented setup (`requirements.txt` / `.env.example`), no hidden manual steps.
-

7. Constraints (from submission instructions)

- GitHub repo must be public.
 - Deck must use the provided RedRob template (PDF export). (*Action item: get the actual template link/file from the hackathon portal — not yet in hand.*)
 - Ranked output must be delivered as `.xlsx`, ≤5MB.
 - Must reference the official dataset provided by RedRob (not yet pulled into this plan — confirm exact field names once accessed, since the schema in the TRD/Backend doc is an informed assumption based on the JD's stated signal categories: profile attributes, career metadata, activity/behavioral signals).
-

8. Success Metrics

Metric	Target
Qualitative ranking sanity (manual spot-check: do top-5 make sense for 3 test JDs)	Pass on all 3
Latency per JD ranking run (pool ≤1000 candidates, embeddings pre-cached)	< 10s

Metric	Target
Explanation quality (human-readable, references specific evidence, no hallucinated facts)	100% of top-10 explanations reference real candidate fields
Submission completeness	Repo + PDF + XLSX all present, correctly formatted
Code cleanliness	Single-command run, README with setup + architecture diagram

9. Risks & Mitigations

Risk	Mitigation
Dataset schema differs from assumptions	Build a thin "adapter" layer (see Backend Schema doc) so field mapping is a config change, not a rewrite
LLM API rate limits / quota exhaustion mid-build (has happened before on Gemini free tier)	Cache all LLM calls (JD parse + explanations) to disk; build a mock/offline mode so the pipeline still runs without live API calls during dev/demo
Hallucinated explanations	Constrain explanation generation to only reference fields actually present in the candidate's structured record (template-fill + light LLM polish, not free-form generation)
Time pressure (deadline July 2)	Implementation plan sequences the riskiest/most-judged parts (ranking engine, explainability) before polish (UI styling, deck)